

Adaptive Queue Length + Fuzzy Logic Congestion Control in Clustered WSN

This document describes the final design for congestion control in a clustered Wireless Sensor Network (WSN). The solution combines **adaptive queue length adjustment** at Cluster Heads (CHs) and **independent fuzzy logic-based congestion detection** using Active Queue Management.

1. Overview of the Proposed Design

Our system contains two independent components:

A. Adaptive Queue Length Mechanism (AQLM)

- Queue length at each CH is *not fixed*.
- It is dynamically adjusted based on:
 - **Packet Loss Ratio (PLR)**
 - **Downstream/Neighbour Cluster Head Capacity**
- AQLM runs **only when PLR exceeds a threshold**.

B. Fuzzy Logic-Based Congestion Detection (FIS + AQM)

- Runs periodically, independent of AQLM.
- Inputs include:
 - Average Queue Length (AQL)
 - Packet Loss Ratio (PLR)
 - Arrival Rate
 - Residual Energy
 - Packet Priority
 - Current Queue Capacity
- Outputs:
 - Drop Probability
 - Throttling/Rate Adjustment
 - Congestion Notification Flag

The two systems **do not control each other internally**, but AQLM affects the queue state which the fuzzy engine reads as an input.

2. Workflow of the System

Step 1: Measure metrics periodically

- PLR updated via EWMA

- AQL calculated from queue occupancy
- Neighbouring CH capacity obtained via control adverts

Step 2: If $PLR > \text{threshold}$, adapt queue length

- Compute desired queue length using PLR
- Scale desired queue using downstream capacity
- Apply smoothing/hysteresis to avoid oscillation
- Update the queue size

Step 3: Fuzzy Logic runs independently

- Reads current queue size and other inputs
- Produces AQM actions
- Handles congestion detection + response

3. Control/Advert Packet Format (Lightweight)

Neighbour CHs periodically advertise their capabilities using a small, piggybacked message.

Advert Packet Structure

```
{
  CH_ID:      uint8,
  service_rate: float (EWMA of pkts/sec),
  avail_buffer: uint8 (free queue slots),
  energy_level: uint8 (optional),
  timestamp:  uint32
}
```

- Sent every $T_{adv} = 2\text{--}5 \text{ seconds}$ - Missing advert for $T_{timeout}$ $\Rightarrow$ assume low capacity

Use in algorithm

- `service_rate` used to compute downstream capacity
- `avail_buffer` used as optional buffer-awareness factor

4. Adaptive Queue Length Algorithm (AQLM)

Parameters

- `Q_min`, `Q_max` – allowed queue limits
- `PLR_th` – threshold at which adaptation begins

- C_{ref} - reference send rate of the CH
- β - smoothing factor
- ΔQ_{min} - minimum effective change
- $step_{max}$ - max queue adjustment per update

Algorithm

```

Every T_meas seconds:
    Measure PLR_sample
    PLR =  $\alpha$ *PLR_sample + (1- $\alpha$ )*PLR

    If PLR > PLR_th and time_since_last_update >= T_hold:
        Q_des_base = Q_min + ((PLR-PLR_th)/(1-PLR_th)) * (Q_max - Q_min)
        C_down = compute_downstream_capacity() # EWMA over neighbour adverts
         $\gamma$  = clamp(C_down / C_ref, 0, 1)

        Q_des = clamp(Q_min +  $\gamma$ *(Q_des_base - Q_min), Q_min, Q_max)

        Q_proposed =  $\beta$ *Q_des + (1- $\beta$ )*Q_current

        If |Q_proposed - Q_current| >=  $\Delta Q_{min}$ :
            change = sign(Q_proposed - Q_current) * min(step_max, |Q_proposed - Q_current|)
            Q_current = Q_current + change
            record_update_time()

```

Notes

- Avoids sudden jumps in queue length
- Prevents cascading congestion due to neighbour overload
- Only triggers under sustained high PLR

5. Fuzzy Inference System (FIS) — Runs Independently

Inputs

- Average Queue Length (AQL)
- Packet Loss Ratio (PLR)
- Arrival Rate
- Packet Priority
- Residual Energy
- **Current Queue Capacity** (output of AQLM)

Outputs

- Drop Probability
- Transmission Rate Adjustment
- Congestion Notification (ECN flag)

Operation

- Runs every **T_FIS** seconds
 - Reads the *current* queue size, but does not influence its calculation
-

6. Neighbour Capacity Calculation

```
For each neighbour i:  
  C_i = EWMA(service_rate_i)  
  
C_down = min_i(C_i)    # conservative  
  
Optionally:  
C_down = weighted_avg(C_i, weights=link_quality)
```

- If no advert received: treat neighbour as low-capacity
 - Ensures stability and avoids cascade failures
-

7. Numerical Example (Short)

- $Q_{\min} = 10$, $Q_{\max} = 100$
 - PLR threshold = 0.10, measured PLR = 0.28
 - Compute base: $Q_{\text{des_base}} = 28$
 - Downstream capacity factor: $\gamma = 0.5$
 - Final: $Q_{\text{des}} = 19$
 - After smoothing $\rightarrow Q_{\text{current}}$ only increases gradually (avoids spikes)
-

8. Advantages of the Design

- **Independent, modular controls** (robust and easy to tune)
 - **Stable queue adaptation** avoids oscillation
 - **Neighbour-aware** adjustment prevents congestion cascades
 - FIS focuses purely on **detecting and responding** to congestion
 - Adaptation triggers only when needed (high PLR)
-

9. Implementation Notes

- EWMA parameters should be tuned depending on simulation
 - Use different timing for AQLM and FIS to maintain independence
 - Limit control packet size to maintain energy efficiency
-

10. Conclusion

This final design provides a novel, hybrid congestion control mechanism for clustered WSNs by: - Dynamically adapting queue length based on PLR and neighbour capacity - Detecting congestion using a separate fuzzy logic AQM engine - Ensuring stability, energy efficiency, and non-cascading operation

It is simple, modular, and directly implementable in simulations such as NS-2/NS-3 or MATLAB.